CAMPUS ATTENDANCE INTEGRITY SYSTEM

Product requirements, security architecture and implementation blueprint

CORE DESIGN GOAL

Make proxy attendance difficult and detectable without making genuine attendance fragile. Security checks must be layered, server-validated and recoverable when a legitimate device or network fails.

DOCUMENT CONTROL	DETAIL
Working name	Campus Attendance Integrity System (CAIS)
Version / date	v1.0 | 9 October 2026
Target environment	AMPICS / college deployment - official institution name and approval to be confirmed
Client applications	Windows Admin/Teacher desktop application + Student Android application
Status	Planning specification; not yet implemented, security-certified or approved for production

Prepared as a shared implementation reference for product, frontend, backend, mobile integration and testing. Decisions marked “confirm” must be resolved with the college before production use.

How to use this blueprint

This document is intended to be the single planning reference shared between the frontend contributor, the backend engineer and the college stakeholder. It separates confirmed product direction from implementation assumptions and open institutional policy choices.

Contents

- 1. Product brief and design principles
- 2. System architecture and deployment model
- 3. Roles, permissions and portal boundaries
- 4. Administration portal requirements
- 5. Teacher portal requirements
- 6. Student Android app requirements
- 7. Attendance lifecycle and decision states
- 8. Anti-proxy threat model and security controls
- 9. Data model and API contract outline
- 10. Reliability, exceptions and manual correction
- 11. Privacy, governance and operational controls
- 12. Technology stack and GitHub collaboration
- 13. Delivery phases and ownership
- 14. Verification plan and release gates
- 15. Decisions to confirm before implementation
- Appendix A. Suggested database entities and API routes
- Appendix B. Reference links

> **READ THIS FIRST**
>
> No individual control - QR, GPS, phone unlock, facial matching or app-integrity checks - can guarantee zero proxy attendance. The system combines independent signals and a controlled human exception path. Its effectiveness must be measured in a real-device pilot.

1. Product brief and design principles

1.1 Problem statement

The existing attendance experience is reported to be glitchy and vulnerable to proxy attendance. The target system must prevent students from easily marking attendance for absent friends, make suspicious attempts detectable, and avoid losing genuine attendance because of camera, location, network or software failures.

1.2 Product objective

> **PRODUCT PROMISE**
>
> One verified student identity. One valid attendance record per class session. Every exception attributable. No “success” UI until the server confirms a durable save.

1.3 Outcomes

- Only students present in the verified academic roster can activate an account.
- Each student uses one active registered Android device at a time, with an audited recovery/replacement process.
- Attendance uses a session-bound, short-lived QR token plus registered-device proof, OS-level local authentication, location assessment and app-level face matching/liveness where approved.
- The backend is authoritative for time windows, session validity, permissions, duplicate prevention and final attendance status.
- Teachers can resolve genuine exceptions, but cannot silently rewrite finalized attendance.
- The application provides clear retry, pending-review and manual-fallback states instead of silently failing.

1.4 Non-goals and claims we will not make

- Do not claim that proxy attendance is mathematically impossible or that any single feature makes the system “100% proxy-proof.”
- Do not use a permanent IP-address lock as the student identity/device anchor. IP can change and may be shared across many people.
- Do not treat a QR scan, GPS match, OS fingerprint/PIN success or ordinary selfie as proof of the student’s identity by itself.
- Do not perform continuous background location tracking for attendance. Location is collected only for an attendance attempt unless a separately approved purpose exists.
- Do not market face/liveness verification as production-grade until the chosen implementation has been evaluated against realistic spoofing and false rejection cases.
- Do not bypass server verification to support offline attendance. Offline exception handling must be explicitly designed and approved.

1.5 Core design principles

Principle	Implementation meaning
Server is authoritative	The client reports inputs and proof; the backend applies the attendance policy and commits the record.
Least privilege	Each role receives only the permissions needed for its tasks; every API request checks role plus resource ownership/assignment. [R4]
Privacy by design	Collect the minimum data for check-in, restrict biometric/location access and define retention before pilot.
Fail safely, recover clearly	A failed check must never silently turn into “Present”; legitimate students receive retry or controlled review.
One shared contract	Windows and Android clients use the same versioned API and status definitions.
Evidence, not assumptions	Security thresholds and liveness claims require real-device test evidence, not UI demos.

2. System architecture and deployment model

The target architecture has two client types and one shared backend. The Windows application provides the Admin and Teacher experiences through role-based navigation. The Android application is dedicated to students and uses native Android capabilities through a Capacitor bridge or an equivalent native implementation.

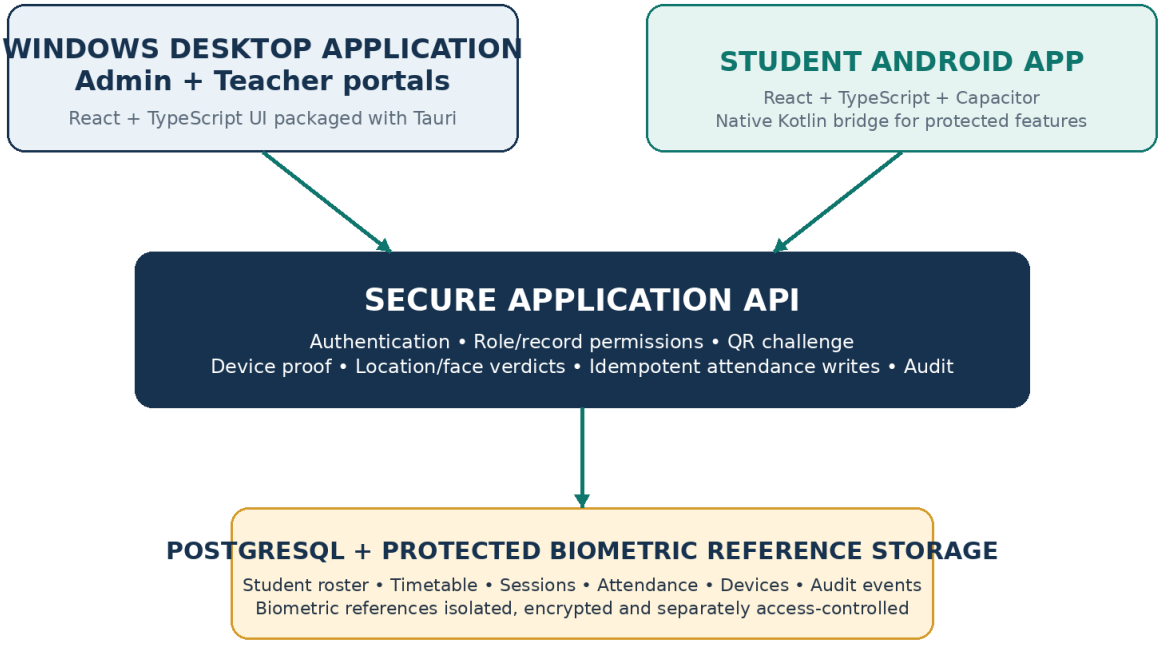


Figure 1. Proposed logical architecture. Biometric references must be isolated and encrypted; the exact storage/provider approach is an approval and implementation decision.

2.1 Recommended implementation stack

Layer	Proposed choice	Why / notes
Admin + Teacher UI	React + TypeScript built in Lovable	Use Lovable for visual system, pages and reusable components; the code still needs review and tests.
Windows packaging	Tauri	Packages the web frontend as a Windows desktop app. Windows installer build and target-device testing are required. [R7]
Student Android UI	React + TypeScript + Capacitor	Enables UI reuse and native Android integration. Camera/location plugins may be insufficient for protected device keys, so custom Kotlin bridge code may be required. [R6]
Native security APIs	Android BiometricPrompt + Android Keystore; verified native location and face/liveness integration	The OS verifies a local authenticator; an enrolled-student face comparison is a separate step. Device-bound signing and fallback behavior need an implementation spike. [R1]
Backend API	Node.js + TypeScript	Centralizes role authorization, challenge validation, atomic attendance writes, idempotency and audit.
Database	PostgreSQL	Stores roster, academic structure, sessions, attendance, device status and audit events.
Source control	GitHub shared repository	One source of truth with protected default branch, feature branches and reviewed pull requests.
Optional Android integrity	Google Play Integrity API	Additional signal for app/device risk; not proof of student identity or classroom presence. [R3]

2.2 Deployment model

- Windows Admin/Teacher software is a client, not the backend server. It communicates with the central API over HTTPS.
- The Android app communicates with the same API; attendance decisions are not made solely inside the app.
- The database and any face-reference store run behind the API. Client apps do not receive unrestricted database credentials.

- For the initial pilot, plan on a hosted/college-approved server and reliable connectivity. A campus-local/offline deployment is a future architecture decision, not an assumed feature.
- The desktop application may not have reliable GPS hardware. Use the Admin-configured classroom location as the primary geofence anchor; a teacher phone/location signal, if available, is secondary.

2.3 Architecture decisions to validate early

Spike	Question to prove before committing
Native device proof	Can the selected Android implementation require local OS authentication and produce a backend-verifiable, challenge-bound device signature?
Face verification	Can the selected SDK/provider do enrolled-reference matching and liveness on intended devices, with documented false acceptance/rejection behavior and privacy terms?
Windows packaging	Can the Lovable-generated project be built cleanly with Tauri, supported on the college PCs, and updated without losing secure API behavior?
Location quality	What location accuracy and room/campus geofence rules are workable in actual classrooms?
Repository sync	Can Lovable import/sync the repository structure without overwriting Antigravity-owned backend/native files?

3. Roles, permissions and portal boundaries

Authentication means proving control of an account/authenticator. Authorization means determining what that authenticated account is allowed to do. The backend must enforce both; hiding a button in the UI is not a security control. [R4]

3.1 Role definitions

Role	Core responsibility	Default access boundary
Administrator	Manage official roster, staff, course structure, rooms, policies, devices, reports and audits.	Institution-level access only for explicitly assigned admin functions; sensitive actions are logged.
Teacher	Run assigned class sessions, monitor check-ins, resolve exceptions, finalize attendance and view class reports.	Only assigned courses/sections and their sessions; no system-wide settings or device credentials.
Student	Register a verified identity/device, attempt own attendance and view own attendance/history/help requests.	Own profile/records only; no access to other students' records or any attendance write/edit API.

3.2 Permission matrix

Action	Admin	Teacher	Student
Import/create student roster	Yes	No	No
Edit official enrolment/course/section	Yes; audited	No	No
Create/manage teacher accounts	Yes	No	No
Assign teacher to section	Yes	No	No
Configure classroom/location policy	Authorized Admin	No	No
Open/close assigned attendance session	Exceptional override only	Yes, assigned sessions	No
Generate live QR	Test/exception only, audited	Yes, assigned session	No
Submit attendance	No impersonation	Manual exception only	Own account only
View attendance list	Institution reports as authorized	Assigned sessions only	Own records only
Manual correction	Restricted review/approval	Reason-required for assigned class	Request only
Register/revoke device	Verified admin recovery	No	Register own device / request recovery
View audit events	Authorized scope	Relevant class events only	Own request outcomes only
Delete or alter audit history	No ordinary privilege	No	No

AUTHORIZATION RULE

Every API request must validate the current account, permission, requested resource and scope. A teacher must not gain access to another teacher's section by editing a URL or request body. Default-deny access and test the rules directly against the API. [R4]

4. Administration portal requirements

The Admin experience is the control plane for academic setup and system governance. It should not be a general "superuser can change anything invisibly" panel.

Screen / module	Required functions	Restrictions / audit
Dashboard	Counts for active students/staff; sessions today; attendance completion; pending review; suspicious/repeated failures; service health.	Metrics respect Admin scope. Avoid exposing face images or raw location points on the dashboard.
Student roster	CSV import preview; validate required fields; reject duplicate enrolment IDs; match college email; view import errors; activate/suspend record.	Every import/update records actor and source. Do not create verified identity from arbitrary student input alone.
Student profile	Name, enrolment ID, institutional email, course, semester, section, active status and verification status.	Changes to identity-critical fields require reason and audit entry. Keep self-edit fields limited to non-authoritative details.
Faculty & assignments	Faculty identity, account status, subject/course assignments, sections and schedule.	Teacher access must reflect assignment changes immediately or within documented policy.
Academic structure	Academic year/term, courses, sections, subjects, timetable, holidays and schedule changes.	Prevent QR/session creation for cancelled or unauthorized schedule entries.
Rooms & location anchors	Registered classroom/campus coordinates, location policy, measurement notes and validity status.	Only authorized Admin can change anchors; changes should be versioned and logged.
Attendance policy	Check-in window; QR lifetime/rotation; late rules; allowed retry count; manual-correction policy; review thresholds.	Do not select location distance thresholds until classroom measurements. Changes affect future sessions unless an explicit migration is approved.
Devices & recovery	View active registration state, revoke lost device credential, initiate replacement workflow and review repeated device changes.	Never display private keys or reusable authentication tokens. Reset events require identity re-verification and audit.
Attendance reports	By course/section/date/student; late/absent/pending/manual-adjustment filters; export.	Exports must be permission-controlled and audited; include only fields needed for reporting.
Audit & review queue	Manual corrections, account recovery, location exceptions, repeated failures and role/policy changes.	Audit logs are append-oriented and restricted; correction is recorded as a new event rather than overwriting history.
Governance settings	Privacy notice/version, retention settings, face-verification mode, exception policy, support contact and release status.	Sensitive policies require institutional approval and should not be silently changed during an open session.

4.1 Admin dashboard status definitions

Status	Meaning
Active / verified	Roster and account checks completed; student is eligible to use attendance.
Pending verification	Roster or college-email confirmation is incomplete. Attendance cannot be accepted as verified.
Suspended / revoked	Account or registered device is disabled. Session submissions from it are denied.
Review required	A human policy action is needed; do not automatically treat an ambiguous signal as proof of cheating.

5. Teacher portal requirements

The Teacher portal is focused on daily class operations, not student administration or system-wide security policy.

Screen / module	Required functions	Restrictions / audit
My timetable	Today/weekly classes; subject; section; room; schedule status; open session action.	Only assigned courses/sections appear, and API checks the assignment independently.
Session setup	Select an eligible scheduled class; confirm course/section/room/time; start or cancel with a reason if permitted.	Do not allow duplicate simultaneous sessions for the same scheduled class without an explicit policy.
QR display	Show high-contrast QR; countdown/expiry indicator; rotate QR; pause/end session; show scanner instructions.	QR minted/validated by server, not generated as a trusted client-only code. Avoid putting student PII in it.
Live attendance	Show confirmed, pending, retry, rejected and manual-review states; filter list; see timestamps and non-sensitive reason codes.	Do not expose face images, biometric templates or precise coordinates in routine views.
Exception resolution	Confirm student is physically present using college-approved method; add/correct attendance with a mandatory reason/category.	No silent edit/delete. Record teacher, time, session, old/new status and reason. High-risk changes may require Admin review.
Finalize session	Review attendance, apply late/absent policy, close session and submit any remaining exception.	Finalized records are protected from ordinary direct edits; later corrections create an adjustment event.
Class reports	View attendance and trends for assigned courses; export approved reports.	Respect student-data access rules and log exports where required.

5.1 Teacher manual adjustment form

Required fields: student, session, action (mark present/correct status), reason category, short explanation, teacher identity from the authenticated session and timestamp. The backend supplies the old value and new value; the client must not be trusted to fabricate audit metadata.

Reason category (proposed)	Required follow-up
Camera / face-verification technical issue	Retry once/within configured policy, then perform identity/presence check per college procedure.
Location unavailable / inaccurate	Retry for fresh location; if still unresolved, use an approved physical confirmation method.
Connectivity / app error	Check submission state to avoid duplicates; create audited manual exception only if no confirmed record exists.
Faculty correction / roster mismatch	Resolve roster or timetable issue; Admin review may be required.
Late arrival or approved exception	Apply the college's configured late/exception policy and label the record accurately.

6. Student Android app requirements

6.1 Screens and actions

Screen	Required behaviour	Must not do
Welcome / sign-in	Enter enrolment number + college email; compare against verified roster; verify mailbox ownership with an approved one-time code or college SSO.	Do not authenticate only because both strings were typed correctly.
Device registration	Register one device-bound key/credential and show account/device status. Require controlled recovery for replacement.	Do not store a reusable device secret in plain app storage or use IP address as permanent identity.
Home	Show next class, attendance status, QR-scan entry point, own attendance history and help.	Do not show other students' data.
Scan QR	Open rear camera or supported scanner; read session token; show course/session context before proceeding.	Do not treat decode success as attendance success.
Location check	Request a fresh location only during check-in; collect accuracy/time; show a simple prompt if permission is unavailable.	Do not track in background for attendance.
Android OS authentication	Show native BiometricPrompt configured for the agreed policy. Use fingerprint/eligible biometric or permitted device credential.	Do not collect fingerprint image, phone PIN or Google password in the app.
App face verification	Open front camera; compare live capture with approved enrolled reference; perform tested liveness if included.	Do not treat Android screen unlock or a single unverified selfie as student-face verification.
Result	Show confirmed / retry / needs faculty review / rejected with clear message and reference.	Do not show "Present" before a server-confirmed commit.
History / correction request	Show own attendance status, session, timestamp, adjustment result and help path.	Student cannot self-edit attendance or remove an audit trail.
Lost phone / support	Request device revoke/recovery, account help or review of disputed attendance.	Do not automatically register a replacement device without re-verification.

6.2 Enrollment and identity binding

- Admin imports or maintains the official student roster, including the institution's authoritative enrolment ID and college email.
- Student enters enrolment ID and email; backend checks a matching active roster record. Email ownership is verified using college-approved email OTP/SSO before activation.
- An authorized method binds the initial real-world student to the account. If facial verification is enabled, biometric reference enrollment must be supervised or otherwise verified against an authoritative college record; an arbitrary selfie must not become the trusted reference.
- The Android app generates a device key using Android Keystore or a suitable native security API. Where available, use a hardware-backed key; registration challenge and public key are stored server-side.
- Only one active device registration is allowed by default. Device replacement requires old-key revocation, re-identity proofing and a logged recovery event.
- Changing IP address, Wi-Fi or mobile carrier must not by itself lock out a legitimate student. Network metadata may be a risk signal, never the core device identity.

6.3 The required phone unlock and face checks are separate

Check	What it proves	What it does not prove
Android BiometricPrompt / device credential	The Android OS accepted an enrolled local biometric or an allowed screen-lock credential for the configured operation. [R1]	It does not prove the enrolled biometric/PIN belongs uniquely to the college student. A friend who knows the device PIN or has a fingerprint enrolled could pass it.
Device-bound challenge signature	The registered device key signed a fresh challenge under the configured local-authentication rules.	It is not, by itself, proof of classroom presence or enrolled face identity.
App-level facial match + liveness	The current face sample matches the approved student reference within evaluated thresholds, with anti-spoof checks.	It is not infallible; model error, presentation attacks, lighting, camera quality and accessibility require testing.
Location proximity	The device reports a fresh location estimate consistent with the approved classroom policy.	It does not prove the correct person is holding the phone; location can be inaccurate or manipulated.

IMPLEMENTATION SPIKE

Before building the full student flow, prove on physical Android devices that the native OS prompt, a device-bound signing key, the configured PIN fallback and a server challenge work as intended. Do not assume that merely receiving “biometric success” in JavaScript creates backend-verifiable device proof. [R1]

7. Attendance lifecycle and decision states

7.1 Session setup

1. Admin maintains an approved timetable, section enrollment and room anchor.
2. Teacher selects an assigned scheduled class. Backend verifies teacher assignment, schedule, room and allowable time window.
3. Teacher opens the session. Backend creates a unique session ID and a signed or server-verifiable QR challenge token with expiry and session binding.
4. Proposed default policy: attendance opens 10 minutes before scheduled class end, as originally suggested. Exact opening, closing and grace rules remain configurable and must be confirmed with the college.
5. QR rotates at a short interval (initial test proposal: 30-60 seconds, to be tuned during pilot). The code can be scanned by many enrolled students while valid; it is not globally single-use. Duplicate protection applies per student per session.

7.2 Student check-in sequence

Step	Backend/client action	Decision result
1. Read QR	Decode session-bound token; backend checks token signature/record, expiry, session, course, schedule and current status.	Invalid/expired/wrong-session → reject with retryable message where appropriate.
2. Locate	Request fresh location with timestamp and reported accuracy; compare to configured room/campus policy. Teacher mobile location, if available, is a secondary signal only.	Within policy → continue; poor accuracy → retry/review; clear mismatch → reject or review according to policy.
3. OS local authentication	Trigger native BiometricPrompt according to policy (biometric and/or device credential). Unlock/sign with registered device credential using a fresh server challenge.	Failure/cancel → controlled retry or help path. No attendance yet.
4. Face match/liveness	Capture live face and verify against the approved enrolled reference. The selected SDK/provider returns a decision plus reason/version; do not rely on basic face detection alone.	Match + liveness pass → continue; uncertain result → retry/review; confident mismatch/failed liveness → reject/review under policy.
5. Submit verification proof	Send session token, challenge nonce, device signature, location metadata and provider verdict/reference as required. Do not trust a client-supplied “passed=true” by itself.	Backend verifies integrity, freshness, authorization and policy thresholds.
6. Atomic attendance write	Inside a database transaction, enforce one record per student-session, store server timestamp and the policy decision, and attach an idempotency key.	Existing same request → return the stored result; conflict → show existing attendance; new valid request → commit.
7. Confirm	Return a server-confirmed result to the app and update the Teacher live view.	Confirmed / Pending review / Retry / Rejected, each with a clear reason code.

7.3 State model

State	Meaning	Can become Present?
NOT_STARTED	No eligible session/check-in is active.	No
CHALLENGE_ISSUED	Server issued a valid QR challenge for an active session.	No
LOCATION_PENDING	Fresh location is being evaluated.	No
DEVICE_AUTH_PENDING	Native OS authentication/device proof is pending.	No
FACE_CHECK_PENDING	Face comparison/liveness is running.	No
READY_TO_COMMIT	All required verification results are accepted by policy.	Only after database commit
CONFIRMED_PRESENT	Backend has committed the attendance record.	Already committed
RETRY_REQUIRED	A recoverable technical or quality issue occurred.	No, until retry passes
MANUAL_REVIEW	A legitimate exception or ambiguous risk requires staff review.	Only after authorized resolution
REJECTED	The request violates a policy rule or fails a required check.	No
SESSION_CLOSED	Session window is closed/finalized.	Only via approved exception path

7.4 QR design rules

- QR content contains only an opaque session/challenge identifier or a signed token; no name, enrolment number, face data or other unnecessary personal data.
- Expiry and session binding are enforced by backend time and stored challenge state, not merely by a countdown in the UI.
- QR rotation narrows the sharing window but cannot alone stop a friend from forwarding a currently valid code.
- Challenge requests and device proofs must be fresh and replay-resistant. The backend must verify nonce/request binding for optional Play Integrity signals as well. [R3]
- Never use one globally single-use QR token for the entire class. The same current QR must allow eligible students to attempt check-in; uniqueness is enforced on each student/session attendance record.

8. Anti-proxy threat model and security controls

Threat modeling defines the attack being addressed, the control, its limitations and the residual risk. It prevents the team from confusing “feature exists” with “attack is prevented.”

Threat / failure case	Primary controls	Residual risk / handling
Student at home gets QR forwarded	Short-lived session QR; device-bound credential; fresh location; OS local auth; face match/liveness.	QR alone is not a presence proof. Mocked location/tampered client risks remain; use server-side challenge verification, app-integrity signal where configured and pilot testing.
Friend is physically in the classroom with absent student’s phone	Registered device, OS authentication and enrolled face comparison with liveness.	A friend may know PIN or have biometric enrolled on the phone. Face comparison is an independent required check if policy enables it.
Friend attempts their own phone using student’s account	One active registered device; account/device cryptographic binding; recovery controls.	A compromised account or approved device replacement can still be abused; log resets and require re-proofing.
Printed photo / phone screen / replayed video	Validated presentation-attack/liveness detection and capture integrity from an evaluated provider.	Basic head-turn prompts alone are not sufficient evidence of robust liveness. Evaluate performance in the actual environment.
Location is inaccurate indoors	Fresh current location with accuracy metadata; calibrated geofence; retry and faculty review.	GPS cannot reliably distinguish every adjacent room or seat. Do not automatically label ambiguous readings as cheating.
Mock location / modified app	Server-side evidence validation; device key; optional Play Integrity; anomaly review.	Integrity signals are risk signals, not a perfect attestation of physical location or identity.
QR / request replay	Expiry; per-request nonce; server challenge binding; idempotency; replay state.	A live challenge can still be forwarded during its valid window; other identity layers remain required.
Student checks in then leaves class	Attendance start check only proves check-in at that moment. Optional random roll-call/recheck or teacher confirmation could be considered.	Requires a clear college rule and student notice; do not add continuous tracking by default.
Teacher changes attendance improperly	Scoped class access; mandatory reason; old/new state; audit event; optional Admin review for late corrections.	Privileged staff misuse is reduced and detectable, not impossible; separate duties for high-risk changes.
Duplicate requests / race condition	Database unique constraint and atomic transaction; idempotency keys.	Test simultaneous double submissions and timeout-after-commit case.
Service or internet outage	Clear pending/error outcome; safe retries; teacher-approved manual fallback.	No silent offline acceptance. True offline operation is a separate future design.

8.1 Security rules (must-have)

- Authorization is enforced at every backend route, including ownership/assignment checks; default deny is the baseline. [R4]
- Session secrets, refresh tokens, private device keys and face data never appear in ordinary logs. Security logs record relevant events without sensitive payloads. [R5]
- QR challenges are generated or signed by the server. The client cannot invent an accepted session ID or override the decision.
- Use HTTPS, secure session management, rate limiting, input validation, secrets management and dependency updates. Keep production secrets out of GitHub and Lovable prompts.
- Use parameterized database access, schema constraints and migration scripts. Verify the authenticated user context in every transaction.
- Account recovery, device reset, face-reference re-enrollment and policy edits are high-risk operations requiring stronger identity proofing and audit.
- Apply generic/consistent error responses that do not leak private student information, face comparison values or internal security logic.
- Run dependency/security scans and test authorization by directly calling APIs, not only through the UI.

9. Data model and API contract outline

9.1 Core data entities

Entity / table	Key fields (proposed)	Purpose / constraints
users	id, role, status, college_email, verified_at	Authentication identity and account lifecycle. Do not duplicate secrets into profile tables.
students	user_id, enrolment_no, name, course_id, semester, section_id, status	Authoritative roster profile; enrolment_no must be unique within the institutional scope.
teachers	user_id, staff_no, status	Staff identity and activity status.
courses / sections / enrollments	course_id, section_id, term_id, student_id, valid_from/to	Academic grouping and eligibility. Prevent overlapping/inconsistent enrollments per policy.
teacher_assignments	teacher_id, section_id, subject_id, valid_from/to	Defines which classes a teacher can operate.
rooms / location_anchors	room_id, approved coordinates/geofence policy, calibration date, status	Configured room/campus reference; version and audit updates.
class_schedules	section, subject, room, teacher, start_at, end_at, status	Timetable source for session eligibility.
attendance_sessions	id, schedule_id, teacher_id, status, opened_at, closed_at, policy_version	Represents one class meeting; prevents unauthorized/duplicate session creation.
qr_challenges	id, session_id, token_hash/reference, issued_at, expires_at, revoked_at	Short-lived session challenges; avoid storing reusable plaintext QR secrets.
device_registrations	id, user_id, public_key, device_label, status, registered_at, revoked_at	Server-side public key and state; private key remains in protected device keystore.
auth_challenges	id/nonce, device_id, purpose, issued_at, expires_at, used_at	Prevents replay and binds proof to a current attendance attempt.
attendance_records	id, session_id, student_id, status, server_timestamp, verification_bundle_ref, source	Unique key on (session_id, student_id) for the current accepted record; commit atomically.
verification_attempts	id, attendance/session ref, check type, outcome code, time, policy version	Stores limited non-sensitive outcomes for troubleshooting and monitoring.
manual_adjustments	id, record_id, old_status, new_status, reason_code, explanation, actor_id, time	Creates attributable corrections without overwriting history.
audit_events	id, actor_id, action, target_type/id, time, correlation_id, redacted_metadata	Append-oriented audit for access, policy, roster, device reset and attendance changes.
biometric_enrollments (isolated)	student reference, provider/template reference, version, status, consent/notice version, dates	Keep templates/references segregated, encrypted and access-limited; minimize raw image retention. Exact design depends on selected provider and college approval.

9.2 API groups (illustrative, not final OpenAPI)

API area	Example routes	Critical validations
Authentication	POST /auth/student/activate; POST /auth/email/verify; POST /auth/session/refresh	Roster match, verified email/SSO, rate limits, account status and safe session rotation.
Devices	POST /devices/register; POST /devices/challenge; POST /devices/revoke-request	Registration/recovery policy, public key validation, device count and challenge freshness.
Academic setup	POST /admin/students/import; POST /admin/assignments; POST /admin/rooms	Admin scope, duplicate data checks, audit, input validation.
Sessions/QR	POST /teacher/sessions; POST /teacher/sessions/{id}/open; POST /sessions/{id}/qr-challenges	Assigned teacher, scheduled time, session state, server issue/expiry.
Attendance	POST /attendance/check-in; GET /sessions/{id}/attendance; POST /attendance/{id}/adjustments	Identity from authenticated principal, challenge/device/location/face checks, unique constraint, idempotency.
Reports/audit	GET /reports/attendance; GET /audit/events; GET /review-queue	Scoped access, export audit, field redaction, pagination and retention controls.

Final route names, schemas and error codes should be written in an OpenAPI contract before parallel frontend/backend work. The examples above are a planning outline, not an implemented API.

10. Reliability, exceptions and manual correction

10.1 Required behavior in common failure cases

Situation	Expected system behavior	Attendance status
Double tap / duplicate submission	Idempotency key and unique database constraint return the original accepted outcome.	One record only.
QR expired before submission	Server returns an expiry reason; app may fetch/scan a new QR within the session window.	No acceptance from expired challenge.
Network lost before server receives request	Show network/pending state; allow safe retry.	Not confirmed.
Server committed but response was lost	Retry returns the already committed result using idempotency/unique key.	One confirmed record.
GPS permission denied / no fix	Explain why location is required; allow one or more policy-limited retries, then faculty review.	Pending/review, not automatically cheating.
Face fails due to light/camera issue	Guided retry and accessibility/support route; repeated ambiguous results go to review.	Not confirmed until accepted by policy.
BiometricPrompt cancelled or PIN fallback	Do not continue to face step/commit unless local authentication and device proof are valid under policy.	Retry or help.
Student phone replaced	Re-verify student; revoke old credential; register new credential; log the reset.	No acceptance from revoked device.
Teacher needs to add presence manually	Check no accepted record exists, check teacher assignment, require reason and capture old/new value.	Manual record with visible source label.
Session cancelled/rescheduled	Invalidate associated QR challenges and block check-ins against stale schedule.	No attendance from cancelled session.
Backend unavailable	Display service unavailable, capture reference for help and use approved teacher process if required.	No false success or silent client-side present.

10.2 Manual corrections workflow

1. Teacher opens the correct assigned session and checks whether an accepted record already exists.
2. Teacher selects the student and a controlled reason category; free-text note is limited to information necessary for the decision.
3. Backend creates an adjustment event containing actor, session, student, old state, new state, reason, time and correlation ID.
4. For finalized sessions, corrections are append-only adjustments, not silent edits. College policy may require Admin approval for late or high-risk changes.
5. The student can see that their record was adjusted and can request review if the college policy allows.
6. Admin review should focus on repeated overrides, late corrections, unusual patterns and policy exceptions; it should not expose biometric images by default.

10.3 Operational monitoring

Metric to observe	Purpose
Check-in success/failure by step	Find which layer causes friction: QR, network, location, device auth or face.
False rejection/manual review rate	Detect whether legitimate students are blocked. Set acceptable targets only after baseline data.
Duplicate/replay rejection count	Confirm replay/duplicate controls function; high volumes may indicate a bug or attack.
Latency and timeout rate	Keep attendance flow responsive at class-change peaks.
Manual override rate by reason/actor/session	Detect recurring technical issues or unusual correction patterns.
Device recovery frequency	Identify account sharing or legitimate device-replacement friction.
Security and API authorization failures	Detect broken access control or client defects while avoiding unnecessary sensitive log data. [R5]

11. Privacy, governance and operational controls

Location and facial verification can involve highly consequential student data. Before real student information is collected, the college must approve the purpose, lawful basis/consent approach, notice, retention, access model, fallback, grievance route and vendor arrangements. This section is a technical planning baseline, not legal advice.

11.1 Data minimization rules

- Collect location only when a student attempts attendance. Avoid continuous background tracking and avoid collecting unrelated movement history.
- Capture only what the chosen face-verification design needs. Prefer minimizing raw-image retention; protect any templates/provider references separately from ordinary academic records.
- Do not log raw QR challenge secrets, device private keys, session tokens, biometric images or unnecessary precise coordinates.
- Limit precise location/face-verification outcomes to roles with a defined operational need. Teacher views should receive reason codes, not biometric payloads.
- Define retention periods for attendance, audit events, verification metadata, device records and biometric references separately. Secure deletion/revocation must be testable.
- Explain what is collected and why, how verification is used, how long data is retained, what alternatives exist and how a student can seek correction or review.
- Offer a controlled alternative for students whose device/camera/biometric setup is unsupported or who encounter repeated false rejection. The alternative should provide comparable review assurance without silently bypassing policy.
- Review the DPDP Act and notified DPDP Rules, applicable commencement timeline and institutional requirements before deployment. [R8, R9]

11.2 Face enrollment and template governance

Control	Requirement
Enrollment source	Approved college record or verified, supervised enrollment process; never trust an arbitrary first selfie as the reference.
Explicit notice and approval	State why face matching/liveness is used and identify the responsible institution/vendor. Obtain the required authorization/consent under the applicable policy before collection. [R10]
Storage	Encrypt data, isolate biometric references, restrict service access, rotate secrets and prevent direct image access from ordinary Admin/Teacher UI.
Comparison provider	Choose only after checking liveness capability, device coverage, security documentation, data-processing terms, offline behavior and evaluation evidence.
Performance evaluation	Measure false accepts and false rejects on representative supported devices, lighting, skin tones, camera quality and accessibility cases.
Retention/revocation	Define deletion on account closure/withdrawal/expiry where applicable; ensure a device/account reset does not silently create a new unverified reference.
Fallback	Failed/uncertain result routes to retry or authorized review; do not treat uncertain biometric output as automatic evidence of misconduct.

11.3 Locations and geofence governance

- Admin-approved room/campus anchors must be measured and versioned. Store who set/changed them and when.
- Choose radius and minimum acceptable accuracy only after testing actual classrooms. A desktop PC may have no reliable GPS, so its location cannot be treated as a guaranteed teacher-location signal.
- An inaccurate location should lead to retry/review, not an automatic cheating verdict.
- Use teacher-device location only as a secondary corroborating signal if the actual teacher device can provide a fresh location with understood accuracy.
- Document how location is used, when it is collected, who can view it and how long it is retained.

11.4 Legal and institutional gate

> **DO NOT SKIP**
>
> Before live biometric or precise-location collection, obtain written college approval and conduct a privacy/security review. Confirm the current legal and rule commencement/applicability position with the institution's responsible authority; this document does not provide legal certification.

12. Technology stack and GitHub collaboration

12.1 Recommended repository layout

```
campus-attendance-system/
├── apps/
│   ├── desktop-portal/      # Admin + Teacher React UI / Tauri shell
│   └── student-android/     # Student UI / Capacitor + native Android code
├── services/
│   └── api/                 # Node.js + TypeScript backend
├── packages/
│   └── shared-contracts/    # Types, request/response schemas, status codes
├── database/
│   └── migrations/          # Versioned schema changes
├── tests/
│   ├── unit/
│   ├── integration/
│   └── security/
└── docs/                    # Requirements, threat model, API contract, ADRs
```

12.2 Lovable + GitHub + Antigravity workflow

Step	Owner / tool	Rule
1. Repository foundation	Project owner creates private GitHub repository and adds collaborators.	Protect default branch; no production secrets or real student data in test commits.
2. UI work	Frontend contributor builds Admin/Teacher interface in Lovable, connected to the shared repo as supported.	Use mock data during UI prototyping. Define routes and components; avoid backend-specific assumptions until API contract exists.
3. Backend work	Backend engineer uses Antigravity to inspect/clone repo and works on the backend/native branches.	Pull current changes before coding; avoid simultaneous edits to the same files.
4. API contract	Team agrees request/response schemas, enums and error codes in docs/shared-contracts.	Frontend and backend do not invent conflicting fields/statuses independently.
5. Review and merge	Feature branch → tests → pull request → review → merge.	Use clear ownership for directories; merge shared changes rather than force-pushing over teammates.
6. Sync and verify	Lovable syncs the linked repository where supported; Antigravity pulls latest changes.	After every merge, run build, type-check, tests and smoke flows. GitHub sync does not guarantee UI and app packages are working.
7. Build artifacts	Tauri build for Windows; Android Studio/SDK builds the app package.	Build on supported environments; sign production packages appropriately and test installation on real devices. [R7, R6]

12.3 Branch and change policy

- Use `frontend`, `backend-api`, `student-android` or short-lived feature branches according to team size; do not make independent conflicting changes to the same default branch.
- Keep frontend screens and backend API types aligned through the shared contracts package and OpenAPI specification.
- Never push `.env`, service-role keys, database passwords, private certificates, production exports or actual biometric data to GitHub.
- AI-generated code must be reviewed; ask Antigravity to preserve the established UI design system unless explicitly asked to redesign.
- For native features, commit and review Kotlin/plugin code alongside the app. A successful React preview does not prove the native prompt, camera, location or Android build works.

13. Delivery phases and ownership

The order below deliberately validates the critical path before expanding features. Dates/durations are not promised here because team size, exact hackathon deadline, tooling readiness and face-provider choice are not yet fixed.

Phase	Deliverable	Exit condition
Phase 0 - Decisions and spikes	Role matrix, attendance policy, database draft, API contract, Android local-auth/device-key proof, face-provider feasibility and room-location measurement plan.	Stakeholders agree what “present”, “late”, “review” and “manual correction” mean. Native security integration is proven on a real test phone.
Phase 1 - Data and access foundation	Roster import, identity verification, role system, course/section assignments, basic device registration.	Unauthorized account/role/resource tests pass. No student can create a verified roster record from self-asserted data.
Phase 2 - Admin/Teacher UI	Lovable UI for Admin and Teacher, session opening, QR display, attendance status, manual adjustment screens.	Screens use shared API types, error states are designed, sample data is clearly non-production.
Phase 3 - Core attendance backend	Scheduled session validation, server QR challenge, device challenge, duplicate protection, atomic save, audit records.	End-to-end demo works without face/GPS complexity and no false success or duplicate records in retry tests.
Phase 4 - Student Android app	Roster activation, device registration, QR scan, native OS authentication, result/history UI.	APK installs on supported devices; native authentication produces expected proof and backend rejects invalid challenges.
Phase 5 - Location and face pilot	Fresh location checks, configured geofence, enrolled face reference, liveness provider/SDK integration and exception path.	Documented test results for genuine, spoofed, inaccurate-location and poor-camera cases; privacy approval obtained.
Phase 6 - Windows packaging and integrated QA	Tauri packaging, install/update verification, full end-to-end testing, monitoring and operational documentation.	Build installs on intended college Windows systems and all release-gate tests pass.
Phase 7 - Controlled pilot	Small approved cohort/class, faculty-assisted monitoring, false rejection review and incident logging.	Stakeholders approve expansion based on measured reliability/security and documented limitations.

13.1 Suggested ownership split

Workstream	Primary owner	Shared responsibility
Product requirements / college workflow	Project owner + faculty contact	Confirm attendance policy, exceptions, data and deployment constraints.
Admin/Teacher UI	Frontend contributor using Lovable	Align to API contract, error states, permissions and accessibility.
Backend / database / audit	Engineering contributor using Antigravity	Enforce policy server-side, migrations, tests, secrets and logs.
Android/native security	Engineering contributor + Android testing support	Native biometric, device key, location, camera and app-integrity integration; physical-device evaluation.
Release and validation	Whole team	Review PRs, run checklist, pilot safely, capture bugs and avoid unverified production claims.

14. Verification plan and release gates

Treat this checklist as an acceptance specification. A checked box means a test was actually executed and evidence was recorded, not that the feature was discussed or is visible in the UI.

14.1 Functional tests

- ☐ Admin imports a roster with valid students and receives actionable errors for invalid/duplicate enrolment IDs.
- ☐ Student account activation requires a matching active roster and verified institutional email/SSO.
- ☐ Teacher can open a scheduled assigned session but cannot open another teacher's session.
- ☐ Active QR can be scanned by multiple eligible students while it is valid; expired, cancelled, revoked and wrong-session challenges are rejected.
- ☐ Successful student check-in is saved once and appears consistently in app, teacher view and report.
- ☐ Student can view only their own attendance history and submit a correction request if policy allows.
- ☐ Teacher can submit manual correction only for an assigned session with required reason and audit entry.
- ☐ Admin device reset revokes the prior device credential and records the reason/actor.
- ☐ Session closure invalidates future normal submissions and routes legitimate late corrections through policy.

14.2 Security tests

- ☐ Call protected APIs directly as Student/Teacher and verify unauthorized routes/resources are denied.
- ☐ Modify student ID, session ID and teacher ID in a request; backend must derive identity from auth and validate assignment, not trust the payload.
- ☐ Replay an expired nonce/device proof or submit the same request in parallel; server must reject replay and preserve a single attendance record.
- ☐ Try a forwarded QR from outside the configured geofence and validate the actual policy outcome; do not assume geofence alone is sufficient.
- ☐ Test a different person using the registered phone, including a known device PIN where policy allows fallback; app-level face check must still run.
- ☐ Test printed photo, screen-replayed photo/video and other approved spoof scenarios against the selected liveness component.
- ☐ Confirm device private keys, tokens, face images and unnecessary precise location do not appear in logs or repository.
- ☐ Test role escalation, account suspension, device revocation, session cancellation and attempted audit modification.

14.3 Reliability / UX tests

- ☐ Network disappears before submit: the UI does not show Present.
- ☐ Server commits but client times out: retry returns the existing outcome without duplication.
- ☐ Poor light, camera permission denial, no location fix, low location accuracy and local authentication cancellation all show clear next steps.
- ☐ Heavy simultaneous scanning does not create duplicate records or allow expired tokens.
- ☐ Desktop app installs and launches on the target Windows environment; Android package installs on actual target phones.
- ☐ UI distinguishes confirmed, retry, pending review, rejected and manual exception states and remains usable on supported screen sizes.

14.4 Release gates

Gate	Required evidence
Gate A - Prototype	Admin → teacher QR → student scan → backend unique record works end-to-end with test data.
Gate B - Security baseline	Role/ownership, expired token, replay, duplicate, manual correction and device-revoke tests pass.
Gate C - Native verification	BiometricPrompt/device credential behavior and cryptographic challenge binding verified on target phones.
Gate D - Biometric/location pilot	Privacy approval plus documented location and liveness/face evaluation, with known failure and fallback behavior.
Gate E - Production readiness	Signed builds, monitoring, backups, recovery/incident process, retention policy, faculty training and institutional approval.

15. Decisions to confirm before implementation

The system design can move forward, but these details must be confirmed rather than silently guessed. The project owner should ask the college contact/admin for answers and record the decision owner/date.

Decision	Recommended starting point	Confirm with
Official institution/authority name and sponsor	Use a working project name until an authorized stakeholder approves the deployment scope.	College/Hackathon organizer
Student roster source and format	Admin-controlled CSV export or official integration; enrolment ID unique within institution.	Admin/IT
College email verification	Official SSO where available; otherwise verified email OTP.	IT/Admin
Attendance window	Configurable; initial proposal opens 10 minutes before scheduled class end.	Faculty/academic policy
QR validity / rotation	Short TTL; test proposal 30-60 seconds, tune under real scan/lighting conditions.	Faculty + engineering
Late/absent/manual override policy	Reason categories and finalization rules defined before pilot.	Academic authority
Room geofence policy	Room/campus anchors measured on site; distance and accuracy threshold set after device testing.	IT/facilities + engineering
Teacher location signal	Use configured classroom as primary reference; teacher phone GPS is optional corroboration. Windows PC location is not assumed reliable.	Faculty/IT
Face reference enrollment	Verified/supervised enrollment using an approved source; never unverified first selfie.	College/admin/privacy authority
Face SDK/provider	Evaluate data location, cost, licensing, supported devices, liveness tests, offline behavior and deletion terms.	Engineering + procurement/privacy
OS local authentication policy	BiometricPrompt + device-bound key; agree when PIN/pattern/password fallback is allowed. Face check remains separate.	Engineering + college policy
Supported Android versions/devices	Pick after checking actual student device distribution and SDK/provider compatibility.	Student cohort/IT
Windows target PCs and distribution	Confirm Windows versions, installer rights, updates and signing expectations.	College IT
Hosting and retention	College-approved environment, backups, incident handling and data retention periods.	IT/privacy authority
Manual review / appeal	Define who resolves ambiguous verification, expected turnaround and student notification.	Academic authority
Privacy/legal approval	Review DPDP Act/Rules applicability, notice/consent, retention, vendor terms and alternative pathway.	Institutional legal/privacy authority

15.1 Immediate next actions

1. Create the private GitHub repository and write this blueprint plus a concise README before parallel coding.
2. Hold a short product decision meeting and lock role permissions, timetable rules, manual correction and enrollment policy.
3. Write the API contract and database migration plan before the frontend and backend branch work diverges.
4. Run a small native Android spike for BiometricPrompt + device-bound challenge signing. This is a critical technical dependency.
5. Run a face-provider feasibility spike before promising automatic liveness in the hackathon demo.
6. Implement the QR attendance core flow first; then add location, device proof and face verification as testable independent layers.

Appendix A. Suggested first API contract

The following request/response shapes are examples to align frontend/backend development. They are not credentials or final schemas. Avoid trusting client-supplied user identity or “passed” booleans.

A.1 Check-in request (conceptual)

```
POST /attendance/check-in
Headers:
  Authorization: Bearer <short-lived-session-token>
  Idempotency-Key: <unique-client-request-id>
Body (conceptual):
{
  "qrChallenge": "<opaque session token>",
  "deviceChallengeId": "<server-issued nonce id>",
  "deviceProof": "<cryptographic signature/proof>",
  "location": {
    "latitude": "<fresh estimate>",
    "longitude": "<fresh estimate>",
    "accuracyMeters": "<reported accuracy>",
    "capturedAt": "<timestamp>"
  },
  "faceVerificationReference": "<provider result reference>"
}
```

Design notes: in actual JSON, latitude, longitude, accuracy and timestamps use typed numeric/date representations. The server binds the authenticated user to the student record; the client does not submit an authoritative student_id. Device proof, face results and integrity tokens need provider-specific validation server-side.

A.2 Response states (conceptual)

status	Meaning	UI response
CONFIRMED	Server committed a valid record.	Show present, time and reference ID.
RETRY_REQUIRED	A recoverable check failed or data is stale.	Explain next step; do not claim attendance was saved.
MANUAL_REVIEW	Policy requires an authorized human decision.	Show pending review and how to contact/track it.
REJECTED	A required policy/security rule was not satisfied.	Give a safe reason code without leaking sensitive security internals.
ALREADY_RECORDED	Same student/session already has a durable record.	Return the existing state without creating a duplicate.
SESSION_CLOSED	The session window is no longer open.	Show the approved late/exception path if one exists.

A.3 Suggested error reason codes

Code	Use
QR_EXPIRED / QR_INVALID	Token not current, invalid or session not active.
SESSION_NOT_ELIGIBLE / SESSION_CLOSED	Wrong schedule/session or policy window closed.
DEVICE_NOT_REGISTERED / DEVICE_REVOKED	Device has no current approved registration.
DEVICE_PROOF_INVALID / CHALLENGE_REPLAYED	Signature/challenge proof is invalid or reused.
LOCATION_UNAVAILABLE / LOCATION_UNCERTAIN / LOCATION_OUTSIDE_POLICY	Location is missing, too inaccurate or outside configured policy.
FACE_RETRY / FACE_MISMATCH / LIVENESS_FAILED	Face verification needs retry, failed to match or failed liveness. These must be backed by an evaluated provider decision.
ALREADY_RECORDED	An accepted record already exists for this student/session.
NETWORK_RETRY / SERVICE_UNAVAILABLE	Safe retry required; result may need status lookup before retrying.
MANUAL_REVIEW_REQUIRED	An authorized review is required by policy.

Appendix B. Reference links

These official or standards-body sources informed the proposed architecture. Re-check version, availability, pricing, applicability and legal status at implementation time. The references support technical design choices; they do not certify the proposed system.

[R1] Android Developers - Show a biometric authentication dialog (BiometricPrompt, BIOMETRIC_STRONG, DEVICE_CREDENTIAL and Android Keystore concepts).
https://developer.android.com/identity/sign-in/biometric-auth?hl=en

[R2] Android Developers - Get the current location (fresh versus stale location estimate and accuracy considerations).
https://developer.android.com/develop/sensors-and-location/location/retrieve-current?authuser=7

[R3] Android Developers - Overview of the Play Integrity API (app/device integrity signals and backend decisions).
https://developer.android.com/google/play/integrity/overview

[R4] OWASP Cheat Sheet Series - Authorization Cheat Sheet (least privilege, default deny and server-side authorization).
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[R5] OWASP Cheat Sheet Series - Logging Cheat Sheet (security event logging and sensitive-data handling).
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[R6] Capacitor Documentation - Android and native plugin support (native Android runtime and custom Kotlin/Java bridging).
https://capacitorjs.com/docs

[R7] Tauri - Windows Installer (Windows installer formats and build considerations).
https://tauri.app/distribute/windows-installer/

[R8] India Code - Digital Personal Data Protection Act, 2023. Review legal applicability and commencement with the institution.
https://www.indiacode.nic.in/indiacode/handle/123456789/22037?view_type=browse

[R9] Ministry of Electronics and Information Technology - Digital Personal Data Protection Rules, 2025 page and associated notices.
https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025-gDOxUjMtQWa?pageTitle=Digit

[R10] NIST SP 800-63-4, Identity Proofing Overview (identity binding, options, liveness and privacy considerations for biometric use).
https://pages.nist.gov/800-63-4/sp800-63a/proofing/

[R11] Capacitor Android Plugin Guide - custom native Java/Kotlin code integration.
https://capacitorjs.com/docs/v6/android/custom-code

> **FINAL STATUS**
>
> This blueprint defines the intended product and engineering behavior. It is not evidence that the system has been built, that its biometric component is secure, or that the college has approved data collection or deployment. Maintain this document with versioned decisions as the implementation changes.